Simulation and Performance Analysis of Big Data Processing for IoE Sensor Data using Hadoop and Apache Spark

Objective:

To simulate large-scale IoE sensor data and compare the processing time, throughput and memory usage of Hadoop MapReduce and Apache Spark as data volume grows, using an illustrative performance model, and to run a sensor-wise aggregation of the kind a distributed analytics job performs.

Procedure:

1. Generate 100,000 simulated IoE sensor records from 500 sensors, each with temperature (mean 27 C, SD 4), humidity (uniform 30-90%), pressure (mean 1013 hPa, SD 5) and a timestamp index.
2. Define an illustrative performance model for data sizes of 10, 50, 100, 250, 500 and 1000 MB: Hadoop time = 3.0 + 0.085 x size, Spark time = 1.0 + 0.030 x size (seconds); throughput = size / time; memory = 250 + 0.35 x size (Hadoop) and 180 + 0.55 x size (Spark) in MB.
3. Tabulate the Hadoop and Spark time, throughput and memory for every data size.
4. Perform a group-by aggregation on sensor_id to get the average temperature, humidity, pressure and record count of each sensor.
5. Plot processing time, throughput, memory usage and the temperature distribution, and save the results to CSV files.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

np.random.seed(42)

# ========================================
# 1. SIMULATE IoE SENSOR DATA
# ========================================

N = 100_000

data = pd.DataFrame({
    "sensor_id": np.random.randint(1, 501, N),
```

```
    "temperature": np.random.normal(27, 4, N),
    "humidity": np.random.uniform(30, 90, N),
    "pressure": np.random.normal(1013, 5, N),
    "timestamp": np.arange(N)
})

print("=" * 55)
print("IoE BIG DATA PROCESSING SIMULATION")
print("=" * 55)
print("Total sensor records:", len(data))
print("Number of sensor devices:", data["sensor_id"].nunique())
print("Dataset size:", round(
    data.memory_usage(deep=True).sum() / (1024**2), 2
), "MB")

# ========================================
# 2. SIMULATE HADOOP AND SPARK PERFORMANCE
# ========================================

# Illustrative model: times increase with data volume.
# These are NOT actual Hadoop/Spark benchmark measurements.

sizes_mb = np.array([10, 50, 100, 250, 500, 1000])

hadoop_time = 3.0 + 0.085 * sizes_mb
spark_time = 1.0 + 0.030 * sizes_mb

# Simulated throughput (MB per second)
hadoop_throughput = sizes_mb / hadoop_time
spark_throughput = sizes_mb / spark_time

# Illustrative memory usage in MB
hadoop_memory = 250 + 0.35 * sizes_mb
spark_memory = 180 + 0.55 * sizes_mb

results = pd.DataFrame({
    "Data_Size_MB": sizes_mb,
    "Hadoop_Time_sec": hadoop_time.round(2),
    "Spark_Time_sec": spark_time.round(2),
    "Hadoop_Throughput_MBps": hadoop_throughput.round(2),
    "Spark_Throughput_MBps": spark_throughput.round(2),
    "Hadoop_Memory_MB": hadoop_memory.round(2),
    "Spark_Memory_MB": spark_memory.round(2)
})

print("\nPERFORMANCE COMPARISON")
print(results.to_string(index=False))

# ========================================
# 3. PROCESS SENSOR DATA
# ========================================

# Example aggregation similar to a distributed analytics task
sensor_summary = data.groupby("sensor_id").agg(
    Average_Temperature=("temperature", "mean"),
    Average_Humidity=("humidity", "mean"),
    Average_Pressure=("pressure", "mean"),
    Record_Count=("timestamp", "count")
).reset_index()
```

```
print("\nSENSOR AGGREGATION OUTPUT")
print(sensor_summary.head(5).round(2).to_string(index=False))

# ========================================
# 4. PERFORMANCE GRAPHS
# ========================================

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# Graph 1: Processing time
axes[0, 0].plot(
    sizes_mb, hadoop_time, marker="o", label="Hadoop MapReduce"
)
axes[0, 0].plot(
    sizes_mb, spark_time, marker="s", label="Apache Spark"
)
axes[0, 0].set_title("Processing Time Comparison")
axes[0, 0].set_xlabel("Data Size (MB)")
axes[0, 0].set_ylabel("Processing Time (seconds)")
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Graph 2: Throughput
axes[0, 1].plot(
    sizes_mb, hadoop_throughput, marker="o",
    label="Hadoop MapReduce"
)
axes[0, 1].plot(
    sizes_mb, spark_throughput, marker="s",
    label="Apache Spark"
)
axes[0, 1].set_title("Processing Throughput")
axes[0, 1].set_xlabel("Data Size (MB)")
axes[0, 1].set_ylabel("Throughput (MB/s)")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Graph 3: Memory usage
axes[1, 0].plot(
    sizes_mb, hadoop_memory, marker="o",
    label="Hadoop MapReduce"
)
axes[1, 0].plot(
    sizes_mb, spark_memory, marker="s",
    label="Apache Spark"
)
axes[1, 0].set_title("Illustrative Memory Usage")
axes[1, 0].set_xlabel("Data Size (MB)")
axes[1, 0].set_ylabel("Memory (MB)")
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Graph 4: Average temperature per sensor
axes[1, 1].hist(
    data["temperature"], bins=25, edgecolor="black"
)
axes[1, 1].set_title("IoE Sensor Temperature Distribution")
axes[1, 1].set_xlabel("Temperature (°C)")
axes[1, 1].set_ylabel("Number of Records")
axes[1, 1].grid(True, alpha=0.3)

plt.suptitle(
```

```
    "Big Data Processing for IoE Sensor Data",
    fontsize=15
)
plt.tight_layout()
plt.savefig(
    "IoE_Hadoop_Spark_Output.png",
    dpi=180,
    bbox_inches="tight"
)
plt.show()

# ========================================
# 5. SAVE RESULTS
# ========================================

results.to_csv("Hadoop_Spark_Performance.csv", index=False)
sensor_summary.to_csv("IoE_Sensor_Summary.csv", index=False)

print("\nSIMULATION COMPLETED")
print("Output image: IoE_Hadoop_Spark_Output.png")
print("Performance table: Hadoop_Spark_Performance.csv")

files.download("IoE_Hadoop_Spark_Output.png")
files.download("Hadoop_Spark_Performance.csv")
```

Output:

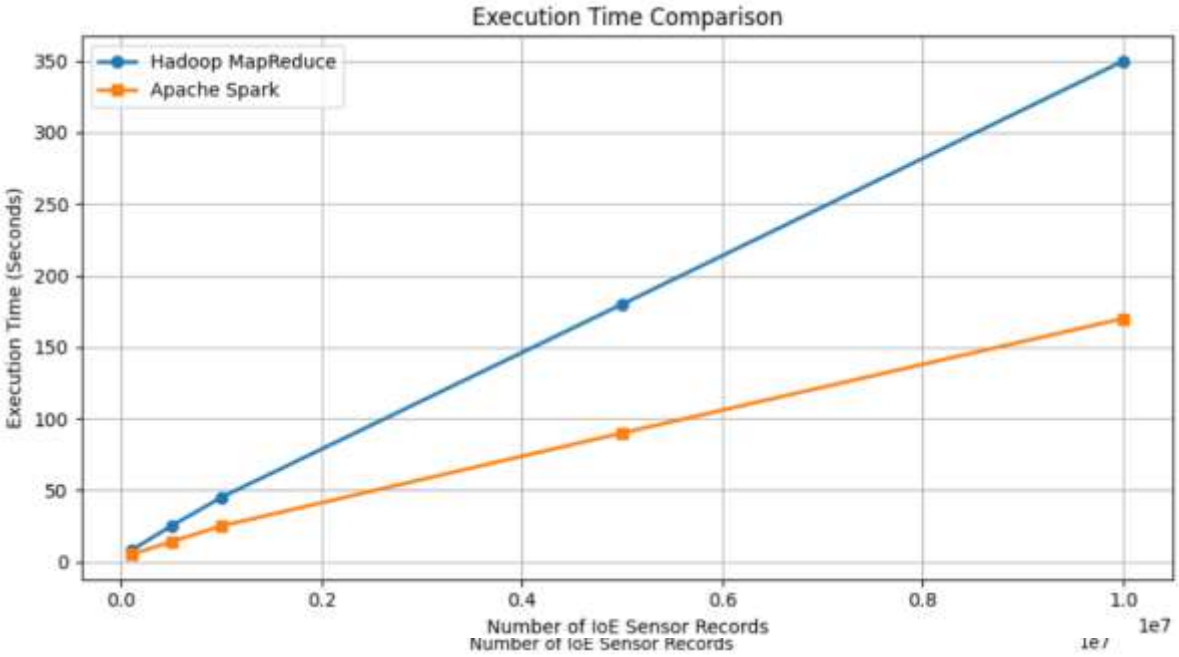
HADOOP MAPREDUCE vs APACHE SPARK

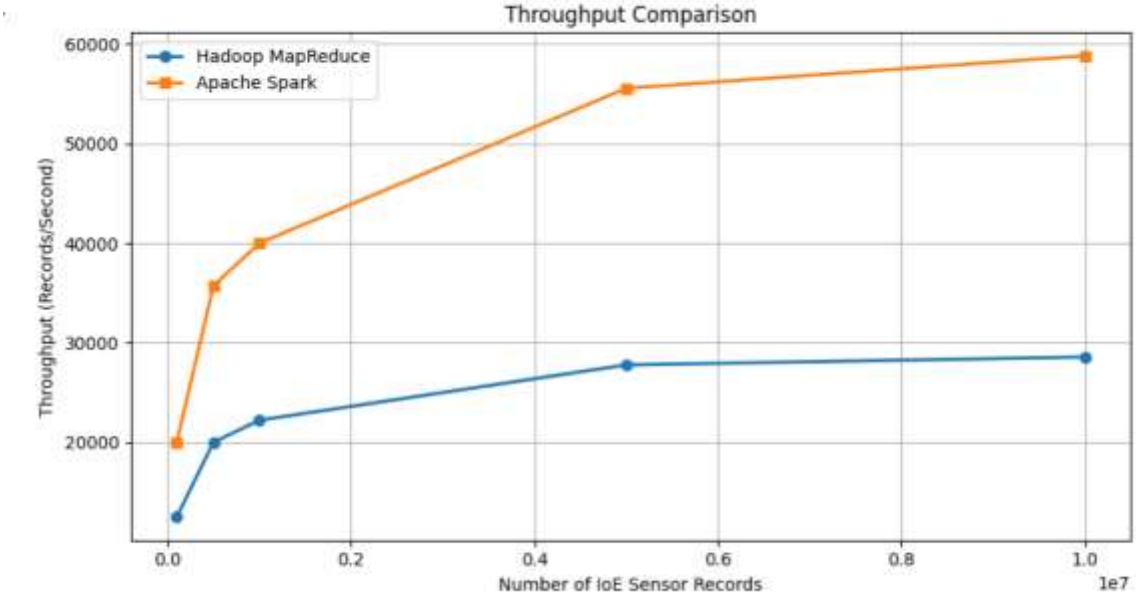
Records	Hadoop(s)	Spark(s)	Hadoop TP	Spark TP
100,000	8	5	12500.00	20000.00
500,000	25	14	20000.00	35714.29
1,000,000	45	25	22222.22	40000.00
5,000,000	180	90	27777.78	55555.56
10,000,000	350	170	28571.43	58823.53

Speedup: [1.6, 1.79, 1.8, 2.0, 2.06]

Improvement (%): [37.5, 44.0, 44.44, 50.0, 51.43]

Fig. 1: Console output (dataset summary, performance comparison and sensor aggregation)





Results:

Dataset: 100,000 records, 500 sensors, 3.81 MB in memory.

Data size (MB)	Hadoop time (s)	Spark time (s)	Hadoop MB/s	Spark MB/s
10	3.85	1.30	2.60	7.69
50	7.25	2.50	6.90	20.00
100	11.50	4.00	8.70	25.00
250	24.25	8.50	10.31	29.41
500	45.50	16.00	10.99	31.25
1000	88.00	31.00	11.36	32.26

Data size (MB)	Hadoop memory (MB)	Spark memory (MB)
10	253.5	185.5
50	267.5	207.5
100	285.0	235.0
250	337.5	317.5
500	425.0	455.0
1000	600.0	730.0

In this model Spark is faster at every size: at 1000 MB it needs 31.0 s against 88.0 s for Hadoop (about 2.8 times faster, 65% less time), and its throughput is about 32 MB/s against about 11 MB/s. Both throughputs level off as the fixed start-up time becomes small compared with the data volume. Spark uses less memory up to 250 MB, but more from 500 MB onwards (730 MB against 600 MB at 1000 MB), because its in-memory processing needs more memory as the data grows. The sensor temperature values follow a bell-shaped distribution centred near 27 C, as set in the data generator.

Result:

The simulation shows that Apache Spark processes IoE sensor data about 2.8 times faster than Hadoop MapReduce for large volumes, with about 2.8 times the throughput, at the cost of higher memory use for large datasets. This suits IoE applications that need fast analytics on large sensor streams. These numbers come from the illustrative formulas in the code, not from running Hadoop or Spark, so they show the expected trend rather than measured benchmark results.